

Microcontrollers Project: Simon Says Game

Bachelor's Degree in Electronic, Robotics and Mechatronics Engineering

Authors: Pablo Martin Sierra
Jaime Esparragoso Vides

Date: January 30, 2026



Universidad de Sevilla

Contents

1	Introduction	2
2	Hardware and Peripheral Configuration	2
3	Code Structure	2
3.1	The Main Loop	2
3.2	State Machine (FSM)	2
4	Detailed Description of Functions and Constants	3
4.1	Definitions and Constants (#define)	3
4.2	Critical Global Variables	3
4.3	Initial Functions	3
4.4	Logic and Algorithm Functions	4
5	Requirements Implementation	4
5.1	General Features and Accessibility (Items 1.x)	4
5.2	Random Generation (Items 2.x)	4
5.3	Interface and Game Flow (Items 3, 4 and 5)	4
5.4	High Scores and Improvements	5
6	Source Code	5

1 Introduction

This report summarizes the development of our project for the course: we implement the classic “Simon Says” game using the MSP430 microcontroller and the BoosterPack MkII. The idea is to meet all the points proposed in the assignment, as specified in the example PDF.

2 Hardware and Peripheral Configuration

For this project we used libraries for the display, so that we could show whatever we wanted on the LCD screen. The rest of the functionality we implemented ourselves in the code:

- **Joystick (ADC10):** We needed to continuously read the stick’s position. We configured the ADC to read channels A0 and A3. The trickiest part here was calibrating the values: the joystick needs tolerance margins so the game doesn’t detect phantom movements.
- **Timers:** We used two:
 - *Timer0:* Our main clock. It gives us an interrupt every 25 ms. This lets us time the rounds without using `delay()`, which is key because it means the microcontroller keeps detecting button presses while it “waits”.
 - *Timer1:* Dedicated exclusively to audio. We use it in compare mode to output a PWM signal to the buzzer. By changing this timer’s frequency on the fly we get the different musical notes.
- **Interrupts:** For buttons S1, S2, and the joystick button, we use interrupts, which drive the behavior carried out by these inputs.
- **LCD Screen:** We use SPI communication, all handled inside the library. It is the most resource-intensive part, so we had to optimize the drawing code so it wouldn’t slow down the game logic.

3 Code Structure

The structure we followed for our code is as follows:

3.1 The Main Loop

The core of the program (`main`) consists of an infinite loop (`while(1)`) that puts the microcontroller into low-power mode (LPM0) on every iteration. The system only “wakes up” through interrupts. The most important interrupt is **Timer0**’s, configured to fire every 25 milliseconds. This interrupt raises a global flag called `tick`.

- If `tick == 0`: The processor sleeps, saving power.
- If `tick == 1`: The main loop runs one cycle of game logic, updates counters, and goes back to sleep.

3.2 State Machine (FSM)

To manage the game logic, we implemented a State Machine controlled by the `estado` variable. The defined states (`estados_t`) are:

1. **BIENVENIDA:** Initial idle state. Waits for the player to press “Start”. Plays a looping melody and checks whether “Symbol Mode” is activated.

2. **MENSAJE_RONDA:** Transition state that clears the screen and announces the current round number to the player.
3. **MAQUINA:** The system steps through the `secuencia` array and lights up the corresponding colors/symbols.
4. **TURNO_JUGADOR:** The system waits for input from the ADC (joystick). It compares the user's input against the stored sequence. If there is an error or the time runs out (*timeout*), we move to **FIN**. If the sequence is completed, we move to **VICTORIA**.
5. **VICTORIA:** We show an animation and success music after clearing a round.
6. **VICTORIA_FINAL:** State reached after clearing all 32 rounds. Shows a star animation and increases the difficulty.
7. **FIN:** "Game Over" screen; shows the final score and allows restarting.

4 Detailed Description of Functions and Constants

A brief expansion on some of the constants and functions declared:

4.1 Definitions and Constants (`#define`)

Macros were mainly used for two purposes: music and graphics.

- **Musical Notes:** We defined the note frequencies (e.g. `#define D0 262`, `#define LA 440`). This makes writing melodies in the code much simpler and more intuitive.
- **Hexadecimal Colors:** The graphics library uses 32-bit values. We defined constants such as `GRAPHICS_COLOR_BRIGHT_GREEN` so we could create our own custom colors.
- **Animation Parameters:** Constants such as `N_VICTF` or `N_ESTRELLAS` control the length of the melodies and the number of little stars on screen.

4.2 Critical Global Variables

Because Interrupt Service Routines (ISRs) cannot return values, we use `volatile` global variables for communication between the interrupts and the main loop:

- `volatile unsigned char tick`: This variable is the system's main clock.
- `volatile unsigned char start`: Indicates whether the joystick button has been pressed.
- `lfsr`: 16-bit shift register for the pseudorandom number generator.

4.3 Initial Functions

These functions isolate the game logic from the MSP430-specific registers, making the code more portable and readable.

`Set_Clk(char VEL)` Configures the timer to run at 1, 8, or 16 MHz. For this game, we set it to 16 MHz.

`inicia_ADC(char canales)` and `lee_ch(char canal)` Configure the ADC and read its value.

`init_buzzer()`, `suen_a_hz()` and `apaga_sonido()` Control Timer1.

- `suen_a_hz` calculates the value of the `TA1CCR0` register based on the desired frequency, to generate a square-wave (50% PWM) signal.
- `apaga_sonido` stops the timer to silence the system.

4.4 Logic and Algorithm Functions

`lfsr_siguiiente()` and `aleatorio_1_4()` Implement the pseudorandom number generator (Requirement 2.3). We use an LFSR (Linear Feedback Shift Register) algorithm that generates a random sequence to play with.

`dibuja_estrella()` A graphics helper function that draws shapes at random (x,y) coordinates for the final-victory animation.

5 Requirements Implementation

Below, we detail how we translated each point of the assignment into our C code:

5.1 General Features and Accessibility (Items 1.x)

- **Colors and Sounds (1.1):** We defined the frequencies (`#define D0 262`, etc.) and the colors in arrays (`color_alto[]` and `color_bajo[]`). This lets us work with them easily.
- **Colorblind Mode (1.2):** In the start routine, we also check whether the player is holding the joystick to the left (`ejex < 100`). If so, we set a flag `modo_simbolos = 1`. From that point on, all drawing logic splits into two paths: it either draws colors, or it draws the characters #, @, \$ and % on a black background.
- **Timing (1.3):** The game's timings are not fixed, but variable. We compute T and T4 based on a `tiempo_base` variable (1000 ms at the start). The state machine uses counters (`tms`) that increment on every timer tick.
- **Visual Feedback (1.4):** To simulate "lighting up", we redraw the same rectangle but change the color from `color_bajo` to `color_alto`.

5.2 Random Generation (Items 2.x)

- **Random Sequence (2.3):** To comply with the restriction, we do not use `stdlib.h` for randomness. We use a 16-bit LFSR (Linear Feedback Shift Register) in the `lfsr_siguiiente()` function. Through XOR operations and bit shifts, we obtain a pseudorandom sequence.
- **Human Seed (2.2):** To make every playthrough unique, the initial seed (`lfsr`) is loaded with the value of `contador_ticks` when the player presses the Start button.
- **Cumulative Sequence (2.5):** We use a single `secuencia[32]` array. In round 1, the machine reads up to index 0; in round 2, up to index 1, and so on. We do not generate a new sequence every round; instead we reuse the initial sequence of 32 terms.

5.3 Interface and Game Flow (Items 3, 4 and 5)

- **Game States:** We split the flow into clear states:
 - BIENVENIDA: The opening melody plays and it waits for the button.
 - MENSAJE_RONDA: Shows the current round number.
 - MAQUINA: Plays back the sequence using the `secuencia` array and the timer.
 - TURNO_JUGADOR: Reads the joystick. If the player takes longer than $2 \cdot T$ (controlled by `tms >= T2`), it jumps to FIN.

- **Score (5.7):** We compute the score step by step. Every correct answer increments the `puntuacion` variable. This means that if you fail partway through a round, you still keep the points for the steps you got right.
- **Hard Mode (5.5):** If the player clears all 32 rounds, the game enters `VICTORIA_FINAL`. If Start is pressed again, we restart the game by setting `modo_rapido = 1`, which halves the base time, making the game twice as fast.

5.4 High Scores and Improvements

We were not able to make progress on this part due to the board's limited resources, since at a certain point it started throwing errors and displaying random colors (video attached). We do have the code for how we implemented the high-score screen to save them, even though it ended up being on an external one.

6 Source Code

Below is the complete code implemented for the project:

```

1  #include <msp430.h>
2  #include <stdio.h>
3  #include <stdint.h>
4
5  #include "gplib.h"
6  #include "Crystalfontz128x128_ST7735.h"
7  #include "HAL_MSP430G2_Crystalfontz128x128_ST7735.h"
8
9  int i=0;
10
11 // Notas musicales y sus frecuencias
12 #define DO 262
13 #define MI 330
14 #define SOL 392
15 #define SI 494
16 #define D02 523
17
18 // Notas extra para la victoria final
19 #define RE 294
20 #define FA 349
21 #define LA 440
22 #define RE2 587
23 #define MI2 659
24 #define SOL2 784
25
26 #define GRAPHICS_COLOR_DARK_YELLOW 0x008B5400
27 #define GRAPHICS_COLOR_BRIGHT_GREEN 0x007FFF00 // verde muy vivo
28
29 Graphics_Context g_sContext;
30
31 // Modo accesible: 0 = colores, 1 = simbolos
32 unsigned char modo_simbolos = 0;
33
34 // Flags de despertar
35 volatile unsigned char tick = 0;
36 volatile unsigned char start = 0;
37 volatile unsigned long contador_ticks = 0;
38
39 // Boton 1 (P1.1)
40 volatile unsigned char boton1 = 0;
41

```

```

42 // Joystick inicial
43 volatile unsigned int ejex = 512;
44 volatile unsigned int ejey = 512;
45
46 // Funcion reloj
47 void Set_Clk(char VEL){
48     BCSCTL2 = SELM_0 | DIVM_0 | DIVS_0;
49     switch(VEL){
50     case 16:
51         if (CALBC1_16MHZ != 0xFF) {
52             __delay_cycles(100000);
53             DCOCTL = 0x00;
54             BCSCTL1 = CALBC1_16MHZ;
55             DCOCTL = CALDCO_16MHZ;
56         }
57         break;
58     case 8:
59         if (CALBC1_8MHZ != 0xFF) {
60             __delay_cycles(100000);
61             DCOCTL = 0x00;
62             BCSCTL1 = CALBC1_8MHZ;
63             DCOCTL = CALDCO_8MHZ;
64         }
65         break;
66     default:
67         if (CALBC1_1MHZ != 0xFF) {
68             DCOCTL = 0x00;
69             BCSCTL1 = CALBC1_1MHZ;
70             DCOCTL = CALDCO_1MHZ;
71         }
72         break;
73     }
74     BCSCTL1 |= XT2OFF | DIVA_0;
75     BCSCTL3 = XT2S_0 | LFX1S_2 | XCAP_1;
76 }
77
78 // Funcion que inicializa el ADC
79 void inicia_ADC(char canales){
80     ADC10CTL0 &= ~ENC;
81     ADC10CTL0 = ADC10ON | ADC10SHT_3 | SREF_0 | ADC10IE;
82     ADC10CTL1 = CONSEQ_0 | ADC10SSEL_0 | ADC10DIV_0 | SHS_0 | INCH_0;
83     ADC10AEO = canales;
84     ADC10CTL0 |= ENC;
85 }
86
87 // Funcion para leer el valor del ADC
88 int lee_ch(char canal){
89     ADC10CTL0 &= ~ENC;
90     ADC10CTL1 &= (0x0fff);
91     ADC10CTL1 |= canal<<12;
92     ADC10CTL0 |= ENC;
93     ADC10CTL0 |= ADC10SC;
94     LPM0;
95     return (ADC10MEM);
96 }
97
98 // Rutina de interrupcion ADC
99 #pragma vector=ADC10_VECTOR
100 __interrupt void ConvertidorAD(void){
101     LPM0_EXIT;
102 }
103
104 // Funcion Timer 25ms

```

```

105 void timer_tick(void){
106     TAOCTL = TASSEL_2 | ID_3 | MC_1; // SMCLK/8 = 2MHz
107     TAOCCRO = 49999; // 25ms
108     TAOCTLO = CCIE;
109 }
110
111 // Rutina de interrupcion Timer
112 #pragma vector=TIMER0_AO_VECTOR
113 __interrupt void Interrupcion_T0(void){
114     tick = 1;
115     contador_ticks++;
116     LPM0_EXIT;
117 }
118
119 // Funcion que inicializa el boton de start (boton del joystick)
120 void boton_start(void){
121     P2DIR &= ~BIT5;
122     P2REN |= BIT5;
123     P2OUT |= BIT5;
124     P2IFG &= ~BIT5;
125     P2IES |= BIT5;
126     P2IE |= BIT5;
127 }
128
129 // Funcion que inicializa el boton 1 (P1.1)
130 void boton_1(void){
131     P1DIR &= ~BIT1;
132     P1REN |= BIT1;
133     P1OUT |= BIT1;
134     P1IFG &= ~BIT1;
135     P1IES |= BIT1;
136     P1IE |= BIT1;
137 }
138
139 // Rutina de interrupcion Boton Joystick
140 #pragma vector=PORT2_VECTOR
141 __interrupt void Interrupcion_P2(void){
142     if(!(P2IN & BIT5)){
143         start = 1;
144     }
145     P2IFG &= ~BIT5;
146     LPM0_EXIT;
147 }
148
149 // Rutina de interrupcion Boton 1 (P1.1)
150 #pragma vector=PORT1_VECTOR
151 __interrupt void Interrupcion_P1(void){
152     if(!(P1IN & BIT1)){
153         boton1 = 1;
154     }
155     P1IFG &= ~BIT1;
156     LPM0_EXIT;
157 }
158
159 // Buzzer desactivado por defecto
160 volatile unsigned char buzzer_activo = 0;
161
162 // Funcion que inicializa el buzzer
163 void init_buzzer(void) {
164     P2DIR |= BIT6;
165     P2SEL &= ~BIT6;
166     P2SEL2 &= ~BIT6;
167     P2OUT &= ~BIT6;

```

```

168     TA1CTL = MC_0;
169     TA1CCTL0 = 0;
170     TA1CCR0 = 1000;
171 }
172
173 // Funcion que activa el sonido
174 void suena_hz(unsigned int hz){
175     unsigned long cuenta_media = 1000000UL / (unsigned long)hz; // 2MHz/(2*hz)
176
177     if(cuenta_media < 10) cuenta_media = 10;
178     if(cuenta_media > 65535) cuenta_media = 65535;
179
180     TA1CTL = TASSEL_2 | ID_3 | MC_1; // SMCLK/8 => 2MHz, UP
181     TA1CCR0 = (unsigned int)(cuenta_media - 1);
182     TA1CCTL0 = CCIE;
183
184     buzzer_activo = 1;
185 }
186
187 // Funcion que desactiva el sonido
188 void apaga_sonido(void){
189     buzzer_activo = 0;
190     TA1CCTL0 &= ~CCIE;
191     TA1CTL = MC_0;
192     P2OUT &= ~BIT6;
193 }
194
195 // Rutina de interrupcion buzzer
196 #pragma vector=TIMER1_A0_VECTOR
197 __interrupt void Interrupcion_T1(void){
198     if(buzzer_activo) P2OUT ^= BIT6;
199     else P2OUT &= ~BIT6;
200 }
201
202 // Funcion que segun el color que elijamos con el joystick, sonara un sonido u otro
203 void sonido_color(unsigned char color){
204     // 1: arriba, 2: derecha, 3: abajo, 4: izquierda
205     switch(color){
206     case 1: suena_hz(D0); break;
207     case 2: suena_hz(MI); break;
208     case 3: suena_hz(SOL); break;
209     case 4: suena_hz(SI); break;
210     default: apaga_sonido(); break;
211     }
212 }
213
214 // LFSR (generador de secuencia pseudoaleatoria)
215 static uint16_t lfsr = 0xACE1u;
216
217 // Funcion para almacenar la semilla
218 void semilla(uint16_t s){
219     if(s == 0) {
220         s = 0xACE1u; // Evitamos semilla a 0
221     }
222     lfsr = s; // Guardamos la semilla
223 }
224
225 // Funcion para calcular los bits de la semilla
226 uint16_t lfsr_siguiete(void){
227     // Calculamos un nuevo bit como el XOR de varios taps
228     uint16_t bit = ((lfsr >> 0) ^ (lfsr >> 2) ^ (lfsr >> 3) ^ (lfsr >> 5)) & 1;
229     lfsr = (lfsr >> 1) | (bit << 15); // Desplazamos el bit y lo metemos por la izquierda
230 }

```

```

231     return lfsr; // Devolvemos el nuevo valor
232 }
233
234 unsigned char aleatorio_1_4(void){
235     return (unsigned char)((lfsr_siguiente() & 0x03) + 1);
236 }
237
238 // Funcion para dibujar una estrella simple (pequena o grande)
239 void dibuja_estrella(unsigned char x, unsigned char y, unsigned char grande, uint32_t color){
240     Graphics_setForegroundColor(&g_sContext, color);
241
242     if(!grande){
243         Graphics_Rectangle p = (Graphics_Rectangle){x, y, x, y};
244         Graphics_fillRectangle(&g_sContext, &p);
245     }
246     else{
247         Graphics_Rectangle h = (Graphics_Rectangle){(int)(x-2), (int)y, (int)(x+2), (int)y};
248         Graphics_Rectangle v = (Graphics_Rectangle){(int)x, (int)(y-2), (int)x, (int)(y+2)};
249         Graphics_fillRectangle(&g_sContext, &h);
250         Graphics_fillRectangle(&g_sContext, &v);
251     }
252 }
253
254 // Funcion principal
255 int main(void){
256     WDTCTL = WDTPW | WDTHOLD; // Stop watchdog-timer
257
258     Set_Clk(16); // Reloj de 16 MHz
259     inicia_ADC(BITO | BIT3); // Joystick A0 y A3
260
261     // Iniciamos lo correspondiente a la pantalla LCD
262     Crystalfontz128x128_Init();
263     Crystalfontz128x128_SetOrientation(LCD_ORIENTATION_UP);
264     Graphics_initContext(&g_sContext, &g_sCrystalfontz128x128);
265     Graphics_setFont(&g_sContext, &g_sFontCm16b);
266     Graphics_setBackgroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
267     Graphics_clearDisplay(&g_sContext);
268
269     init_buzzer(); // Inicializamos el buzzer
270     timer_tick(); // Inicializamos el timer
271     boton_start(); // Inicializamos el boton de start
272     boton_1(); // Inicializamos el boton 1
273
274     __bis_SR_register(GIE);
275
276     // Dibujo de los elementos del juego
277     Graphics_Rectangle boton[4]; // 4 rectangulos del simon dice
278
279     unsigned int xC = 64; // Centro X
280     unsigned int yC = 70; // Centro Y
281     unsigned int c = 26; // Tamano del hueco central
282     unsigned int t = 30; // Grosor del anillo
283
284     // Coordenadas del hueco central
285     unsigned int x0 = xC - c/2;
286     unsigned int x1 = xC + c/2;
287     unsigned int y0 = yC - c/2;
288     unsigned int y1 = yC + c/2;
289
290     // Coordenadas del contorno exterior del anillo
291     unsigned int xL = xC - (c/2 + t);
292     unsigned int xR = xC + (c/2 + t);
293     unsigned int yT = yC - (c/2 + t);

```

```

294     unsigned int yB = yC + (c/2 + t);
295
296     unsigned int gJ = 3; // Espacio de separacion entre rectangulos del anillo
297
298     // Coordenadas de los rectangulos
299     boton[0] = (Graphics_Rectangle){(int)(x0 + gJ),(int)yT,(int)(xR),(int)y0}; // Amarillo (
300     arriba)
301     boton[1] = (Graphics_Rectangle){(int)x1,(int)(y0 + gJ),(int)xR,(int)yB}; // Azul (derecha)
302     boton[2] = (Graphics_Rectangle){(int)(xL),(int)y1,(int)(x1 - gJ),(int)yB}; // Verde (abajo)
303     boton[3] = (Graphics_Rectangle){(int)xL, (int)yT, (int)x0, (int)(y1 - gJ)}; // Rojo (
304     izquierda)
305
306     // Marco exterior
307     Graphics_Rectangle marco = (Graphics_Rectangle){2, 2, 125, 125};
308
309     // Barra de porcentaje
310     Graphics_Rectangle barra = (Graphics_Rectangle){12, 8, 110, 18};
311     Graphics_Rectangle caja_pct = (Graphics_Rectangle){112, 8, 120, 18};
312
313     // Animacion victoria
314     Graphics_Rectangle barra1 = (Graphics_Rectangle){44,90,52,114};
315     Graphics_Rectangle barra2 = (Graphics_Rectangle){58,90,66,114};
316     Graphics_Rectangle barra3 = (Graphics_Rectangle){72,90,80,114};
317     Graphics_Rectangle marco_eq = (Graphics_Rectangle){36,86,88,118};
318
319     // Array colores (arriba, derecha, abajo, izquierda)
320     uint32_t color_alto[4], color_bajo[4];
321
322     // Array de simbolos
323     static const int8_t simbolos[4][2] = { {'#',0}, {'@',0}, {'$',0}, {'%',0} };
324
325     // Asignamos los colores al array de colores iluminados
326     color_alto[0] = GRAPHICS_COLOR_YELLOW;
327     color_alto[1] = GRAPHICS_COLOR_BLUE;
328     color_alto[2] = GRAPHICS_COLOR_BRIGHT_GREEN;
329     color_alto[3] = GRAPHICS_COLOR_RED;
330
331     // Asignamos los colores al array de colores apagados
332     color_bajo[0] = GRAPHICS_COLOR_DARK_YELLOW;
333     color_bajo[1] = GRAPHICS_COLOR_DARK_BLUE;
334     color_bajo[2] = GRAPHICS_COLOR_DARK_GREEN;
335     color_bajo[3] = GRAPHICS_COLOR_DARK_RED;
336
337     // Juego
338     unsigned char secuencia[32]; // array que almacenara las 32 rondas de juego
339
340     unsigned int ronda = 1; // Empezamos en la ronda 1
341     unsigned int puntuacion = 0; // La puntuacion inicial es 0
342
343     // Definimos las variables de los tiempos
344     unsigned int tiempo_base = 1000; // ms
345     unsigned int tiempo = 1000; // ms
346     unsigned char modo_rapido = 0;
347
348     unsigned int T = (unsigned int)(tiempo / 25);
349     if(T < 2) T = 2;
350     unsigned int T4 = (unsigned int)(T / 4);
351     if(T4 < 1) T4 = 1;
352     unsigned int T2 = (unsigned int)(2 * T);
353
354     // Defino los estados de la FSM
355     typedef enum {
356         BIENVENIDA=0, MENSAJE_RONDA, MAQUINA, TURNO_JUGADOR,

```

```

355     VICTORIA, VICTORIA_FINAL, FIN
356 } estados_t;
357
358 estados_t estado = BIENVENIDA;
359
360 // Variables auxiliares de la FSM
361 unsigned char sub_maquina = 0;
362 unsigned int paso_maquina = 0;
363 unsigned int paso_jugador = 0;
364 unsigned int tms = 0;
365 unsigned char estado_joystick = 0;
366 unsigned char seleccion = 0;
367 unsigned int tflash = 0;
368
369 // Variables victoria
370 unsigned int t_vict = 0;
371 unsigned int paso_vict = 0;
372 unsigned char anim = 0;
373
374 // Variables victoria final
375 unsigned int t_victf = 0;
376 unsigned int paso_victf = 0;
377 unsigned char animf = 0;
378
379 // Melodia victoria final
380 #define N_VICTF 18
381 static const unsigned int melodia_victf[N_VICTF] = {
382     D0,MI,SOL,D02,RE2,MI2,RE2,D02,
383     0,SOL,LA,SOL,MI,FA,MI,RE,D0,0
384 };
385 static const unsigned char duracion_victf[N_VICTF] = {
386     4,4,4,6,4,4,4,6,
387     3,4,4,4,4,4,4,8,8
388 };
389
390 // Melodia bienvenida
391 #define N_BIENV 16
392 static const unsigned int melodia_bienv[N_BIENV] = {
393     D0,0,MI,0,SOL,0,D02,0,
394     LA,0,SOL,0,FA,0,MI,0
395 };
396 static const unsigned char duracion_bienv[N_BIENV] = {
397     4,2,4,2,4,2,6,3,
398     4,2,4,2,4,2,6,6
399 };
400
401 unsigned int t_bienv = 0;
402 unsigned int paso_bienv = 0;
403
404 // Estrellas victoria final
405 #define N_ESTRELLAS 12
406 unsigned char star_x[N_ESTRELLAS];
407 unsigned char star_y[N_ESTRELLAS];
408 unsigned char star_big[N_ESTRELLAS];
409 unsigned char star_on[N_ESTRELLAS];
410
411 // Bucle infinito
412 while(1){
413     LPMO; // Dormimos al micro
414
415     // Entramos cada 25ms
416     if(tick){
417         tick = 0;

```

```

418         tms++;
419
420         if(boton1){
421             boton1 = 0;
422             apaga_sonido();
423             estado = VICTORIA_FINAL;
424             tms = 0;
425         }
426
427         switch(estado){
428             // Pantalla de bienvenida
429             case BIENVENIDA:
430                 if(tms == 1){
431                     apaga_sonido(); // Buzzer desactivado
432
433                     tiempo = tiempo_base;
434                     modo_rapido = 0;
435
436                     Graphics_clearDisplay(&g_sContext);
437                     Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
438                     Graphics_drawStringCentered(&g_sContext, "SIMON DICE", -1, 64, 50,
TRANSPARENT_TEXT);
439                     Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 75,
TRANSPARENT_TEXT);
440
441                     t_bienv = 0;
442                     paso_bienv = 0;
443                 }
444
445                 t_bienv++;
446
447                 if(t_bienv == 1){
448                     if(melodia_bienv[paso_bienv] == 0){
449                         apaga_sonido();
450                     }
451                     else{
452                         suena_hz(melodia_bienv[paso_bienv]);
453                     }
454                 }
455
456                 if(t_bienv >= duracion_bienv[paso_bienv]){
457                     t_bienv = 0;
458                     paso_bienv++;
459                     if(paso_bienv >= N_BIENV){
460                         paso_bienv = 0;
461                     }
462                 }
463
464                 // Si la flag de start esta a 1, iniciamos el juego
465                 if(start){
466                     start = 0;
467                     apaga_sonido();
468
469                     // Modo accesible
470                     ejex = (unsigned int)lee_ch(0);
471                     if(ejex < 100) {
472                         modo_simbolos = 1; // Joystick a la izquierda
473                     }
474                     else {
475                         modo_simbolos = 0; // Centro/derecha colores
476                     }
477
478                     // Se decide la semilla aleatoria

```

```

479         semilla((uint16_t)(contador_ticks ^ 0xBEEF));
480         for(i=0;i<32;i++){
481             secuencia[i] = aleatorio_1_4();
482         }
483
484         // Reset del juego
485         ronda = 1;
486         puntuacion = 0;
487         paso_maquina = 0;
488         paso_jugador = 0;
489         sub_maquina = 0;
490
491         // Duracion de la secuencia de la maquina
492         T = (unsigned int)(tiempo / 25);
493         if(T < 2) T = 2;
494
495         // Tiempo entre pasos de la secuencia
496         T4 = (unsigned int)(T / 4);
497         if(T4 < 1) T4 = 1;
498
499         // Duracion de la secuencia del jugador
500         T2 = (unsigned int)(2 * T);
501
502         estado = MENSAJE_RONDA;
503         tms = 0;
504     }
505     break;
506
507     // Estado de mensaje entre ronda y ronda
508     case MENSAJE_RONDA:
509         if(tms == 1){
510             char cad[20];
511             apaga_sonido(); // Buzzer desactivado
512
513             // Dibujamos el texto de por que ronda vamos
514             Graphics_clearDisplay(&g_sContext);
515             sprintf(cad, "RONDA %d", ronda);
516             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
517             Graphics_drawStringCentered(&g_sContext, (int8_t*)cad, -1, 64, 55,
TRANSPARENT_TEXT);
518             Graphics_drawStringCentered(&g_sContext, "MIRA...", -1, 64, 80,
TRANSPARENT_TEXT);
519
520             paso_maquina = 0;
521             sub_maquina = 0;
522         }
523
524         // Tiempo de espera la pantalla de ronda
525         if(tms >= (800/25)){
526             Graphics_clearDisplay(&g_sContext); // Limpiamos la pantalla
527
528             // Dibujamos el marco
529             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
530             Graphics_drawRectangle(&g_sContext, &marco);
531
532             // Dibujamos la barra de progreso de la ronda
533             Graphics_drawRectangle(&g_sContext, &barra);
534             Graphics_drawRectangle(&g_sContext, &caja_pct);
535             Graphics_drawString(&g_sContext, "%", -1, 122, 7, TRANSPARENT_TEXT);
536
537             // Dibujamos los 4 rectangulos de color o simbolos
538             for(i=0;i<4;i++){
539                 if(!modo_simbolos){

```

```

540         Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
541         Graphics_fillRectangle(&g_sContext, &boton[i]);
542     }
543     else{
544         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
545         Graphics_fillRectangle(&g_sContext, &boton[i]);
546         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
547         Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[i], -1,
548             (boton[i].xMin + boton[i].xMax)/2,
549             (boton[i].yMin + boton[i].yMax)/2, TRANSPARENT_TEXT);
550     }
551 }
552
553 estado = MAQUINA;
554 tms = 0;
555 }
556 break;
557
558 case MAQUINA:
559     // Si ya se han mostrado todos los pasos, pasamos al turno del jugador
560     if(paso_maquina >= ronda){
561         apaga_sonido(); // Buzzer desactivado
562
563         // Redibujamos estado inactivo
564         for(i=0; i<4; i++){
565             if(!modo_simbolos){
566                 Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
567                 Graphics_fillRectangle(&g_sContext, &boton[i]);
568             }
569             else{
570                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
571                 Graphics_fillRectangle(&g_sContext, &boton[i]);
572                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
573                 Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[i], -1,
574                     (boton[i].xMin + boton[i].xMax)/2,
575                     (boton[i].yMin + boton[i].yMax)/2, TRANSPARENT_TEXT);
576             }
577         }
578
579         paso_jugador = 0;
580         estado_joystick = 0;
581         estado = TURNO_JUGADOR;
582         tms = 0;
583         break;
584     }
585
586     // Subestado para iluminar color
587     if(sub_maquina == 0){
588         if(tms == 1){
589
590             unsigned char color = secuencia[paso_maquina];
591             unsigned char color_aux = color - 1; // Array de 0 a 3
592
593             if(!modo_simbolos){
594                 Graphics_setForegroundColor(&g_sContext, color_alto[color_aux]);
595                 Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
596             }
597             else{
598                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
599                 Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
600                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
601                 Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[color_aux
], -1,

```

```

602         (boton[color_aux].xMin + boton[color_aux].xMax)/2,
603         (boton[color_aux].yMin + boton[color_aux].yMax)/2,
TRANSPARENT_TEXT);
604     }
605
606     sonido_color(color);
607
608     // Barra progreso maquina (relleno rojo)
609     Graphics_Rectangle interior = barra;
610     interior.xMin += 1; interior.yMin += 1;
611     interior.xMax -= 1; interior.yMax -= 1;
612
613     Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
614     Graphics_fillRectangle(&g_sContext, &interior);
615
616     unsigned int W = interior.xMax - interior.xMin;
617     unsigned int relleno = (unsigned int)((unsigned long)W * (paso_maquina
+1) / ronda);
618
619     if(relleno > 0){
620         Graphics_Rectangle f = interior;
621         f.xMax = f.xMin + relleno;
622         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_RED);
623         Graphics_fillRectangle(&g_sContext, &f);
624     }
625
626     // Tras T segundos
627     if(tms >= T){
628         unsigned char color = secuencia[paso_maquina];
629         unsigned char color_aux = color - 1;
630
631         if(!modo_simbolos){
632             Graphics_setForegroundColor(&g_sContext, color_bajo[color_aux]);
633             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
634         }
635         else {
636             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
637             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
638             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
639             Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[color_aux
], -1,
640             (boton[color_aux].xMin + boton[color_aux].xMax)/2,
641             (boton[color_aux].yMin + boton[color_aux].yMax)/2,
TRANSPARENT_TEXT);
642         }
643
644         apaga_sonido(); // Buzzer desactivado
645         sub_maquina = 1; // Pasamos al subestado de pausa entre colores
646         tms = 0;
647     }
648 }
649 else {
650     if(tms >= T4){
651         paso_maquina++; // Siguiente paso
652         sub_maquina = 0;
653         tms = 0;
654     }
655 }
656 break;
657
658 case TURNNO_JUGADOR: {
659     // Si se acaba el tiempo, game over
660     if(tms >= T2){

```

```

661         estado = FIN;
662         tms = 0;
663         apaga_sonido();
664         break;
665     }
666
667     if(tms == 1){
668         // Inicializamos los botones apagados
669         for(i=0; i<4; i++){
670             if(modosimbolos){
671                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
672                 Graphics_fillRectangle(&g_sContext, &boton[i]);
673                 Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
674                 Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[i], -1,
675                     (boton[i].xMin + boton[i].xMax)/2,
676                     (boton[i].yMin + boton[i].yMax)/2, TRANSPARENT_TEXT);
677             }
678             else{
679                 Graphics_setForegroundColor(&g_sContext, color_bajo[i]);
680                 Graphics_fillRectangle(&g_sContext, &boton[i]);
681             }
682         }
683     }
684
685     // Leer joystick
686     ejex = (unsigned int)lee_ch(0);
687     ejey = (unsigned int)lee_ch(3);
688
689     int dx = (int)ejex - 512;
690     int dy = (int)ejey - 512;
691
692     // Zona muerta
693     if(dx < 100 && dx > -100 && dy < 100 && dy > -100){
694         estado_joystick = 0;
695     }
696
697     unsigned char elegido = 0;
698
699     if(estado_joystick == 0){
700         if(!(dx < 100 && dx > -100 && dy < 100 && dy > -100)) {
701             estado_joystick = 1;
702             int adx = (dx<0)?-dx:dx;
703             int ady = (dy<0)?-dy:dy;
704
705             if(adx > ady) {
706                 if(dx > 0) elegido = 2; // Derecha
707                 else elegido = 4; // Izquierda
708             }
709             else {
710                 if(dy > 0) elegido = 1; // Arriba
711                 else elegido = 3; // Abajo
712             }
713         }
714     }
715
716     // Si se ha elegido algun color
717     if(elegido != 0) {
718         unsigned char esperado = secuencia[paso_jugador];
719         {
720             unsigned char color_aux = elegido - 1;
721             if(!modosimbolos) {
722                 Graphics_setForegroundColor(&g_sContext, color_alto[color_aux]);
723                 Graphics_fillRectangle(&g_sContext, &boton[color_aux]);

```

```

724         }
725         else {
726             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
727             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
728             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
729             Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[color_aux
], -1,
730                                     (boton[color_aux].xMin + boton[color_aux].xMax)/2,
731                                     (boton[color_aux].yMin + boton[color_aux].yMax)/2,
TRANSPARENT_TEXT);
732         }
733
734         sonido_color(elegido);
735         seleccion = elegido;
736         tflash = 0;
737     }
738
739     // Si el color elegido es el correcto
740     if(elegido == esperado){
741         puntuacion++;
742         paso_jugador++;
743
744         // Barra progreso jugador (relleno blanco)
745         Graphics_Rectangle interior = barra;
746         interior.xMin += 1; interior.yMin += 1;
747         interior.xMax -= 1; interior.yMax -= 1;
748
749         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
750         Graphics_fillRectangle(&g_sContext, &interior);
751
752         unsigned int W = interior.xMax - interior.xMin;
753         unsigned int relleno = (unsigned int)((unsigned long)W * (paso_jugador
/ ronda);
754
755         if(relleno > 0){
756             Graphics_Rectangle f = interior;
757             f.xMax = f.xMin + relleno;
758             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
759             Graphics_fillRectangle(&g_sContext, &f);
760         }
761
762         // Si completa la ronda
763         if(paso_jugador >= ronda){
764             estado = VICTORIA;
765             tms = 0;
766             t_vict = 0;
767             paso_vict = 0;
768             anim = 0;
769             apaga_sonido();
770         }
771         else{
772             tms = 0;
773         }
774     }
775     else {
776         estado = FIN; // ERROR
777         tms = 0;
778     }
779
780     if(seleccion != 0){
781         tflash++;
782         if(tflash >= (T/2)) {
783             unsigned char color_aux = seleccion - 1;

```

```

784         if(!modo_simbolos) {
785             Graphics_setForegroundColor(&g_sContext, color_bajo[color_aux]);
786             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
787         }
788         else {
789             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
790             Graphics_fillRectangle(&g_sContext, &boton[color_aux]);
791             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
792             Graphics_drawStringCentered(&g_sContext, (int8_t*)simbolos[color_aux
],-1,
793                                     (boton[color_aux].xMin + boton[color_aux].xMax)/2,
794                                     (boton[color_aux].yMin + boton[color_aux].yMax)/2,
TRANSPARENT_TEXT);
795         }
796
797         apaga_sonido();
798         seleccion = 0;
799     }
800 }
801 break;
802 }
803
804 // Estado de ronda completada
805 case VICTORIA:
806     if(tms == 1) {
807         Graphics_clearDisplay(&g_sContext);
808         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
809         Graphics_drawStringCentered(&g_sContext, "RONDA", -1, 64, 45,
TRANSPARENT_TEXT);
810         Graphics_drawStringCentered(&g_sContext, "SUPERADA", -1, 64, 65,
TRANSPARENT_TEXT);
811
812         Graphics_drawRectangle(&g_sContext, &marco_eq);
813         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
814         Graphics_fillRectangle(&g_sContext, &barra1);
815         Graphics_fillRectangle(&g_sContext, &barra2);
816         Graphics_fillRectangle(&g_sContext, &barra3);
817
818         paso_vict = 0;
819         t_vict = 0;
820         anim = 0;
821     }
822
823     t_vict++;
824
825     // Musica simple de victoria
826     if(paso_vict == 0) suena_hz(D0);
827     if(paso_vict == 1) apaga_sonido();
828     if(paso_vict == 2) suena_hz(MI);
829     if(paso_vict == 3) apaga_sonido();
830     if(paso_vict == 4) suena_hz(SOL);
831     if(paso_vict == 5) apaga_sonido();
832     if(paso_vict == 6) suena_hz(D02);
833     if(paso_vict == 7) apaga_sonido();
834     if(paso_vict == 8) suena_hz(SOL);
835     if(paso_vict == 9) apaga_sonido();
836     if(paso_vict == 10) suena_hz(D02);
837     if(paso_vict == 11) apaga_sonido();
838
839     if((paso_vict % 2) == 0) {
840         if(t_vict >= 6) { t_vict = 0; paso_vict++; }
841     }
842     else {

```

```

843         if(t_vict >= 2) { t_vict = 0; paso_vict++; }
844     }
845
846     // Animacion ecualizador
847     if((tms % 2) == 0){
848         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_BLACK);
849         Graphics_fillRectangle(&g_sContext, &barra1);
850         Graphics_fillRectangle(&g_sContext, &barra2);
851         Graphics_fillRectangle(&g_sContext, &barra3);
852
853         unsigned char h1=8, h2=18, h3=12;
854         if(anim==0) { h1=8; h2=18; h3=12; }
855         if(anim==1) { h1=18; h2=10; h3=16; }
856         if(anim==2) { h1=12; h2=16; h3=8; }
857         if(anim==3) { h1=16; h2=8; h3=18; }
858
859         if((paso_vict % 2) == 1){ h1 = 6; h2 = 6; h3 = 6; }
860
861         Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
862         Graphics_Rectangle b1 = barra1; b1.yMin = (unsigned int)(barra1.yMax - h1);
863         Graphics_Rectangle b2 = barra2; b2.yMin = (unsigned int)(barra2.yMax - h2);
864         Graphics_Rectangle b3 = barra3; b3.yMin = (unsigned int)(barra3.yMax - h3);
865
866         Graphics_fillRectangle(&g_sContext, &b1);
867         Graphics_fillRectangle(&g_sContext, &b2);
868         Graphics_fillRectangle(&g_sContext, &b3);
869
870         anim++;
871         if(anim >= 4) anim = 0;
872     }
873
874     // Fin animacion
875     if(paso_vict >= 12) {
876         apaga_sonido();
877         ronda++;
878         if(ronda > 32) {
879             estado = VICTORIA_FINAL;
880             tms = 0;
881             break;
882         }
883         paso_maquina = 0;
884         sub_maquina = 0;
885         paso_jugador = 0;
886         seleccion = 0;
887         estado = MENSAJE_RONDA;
888         tms = 0;
889     }
890     break;
891
892     // Estado de victoria final
893     case VICTORIA_FINAL:
894         if(tms == 1){
895             apaga_sonido();
896             Graphics_clearDisplay(&g_sContext);
897
898             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
899             Graphics_drawStringCentered(&g_sContext, "VICTORIA!", -1, 64, 45,
TRANSPARENT_TEXT);
900             Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 95,
TRANSPARENT_TEXT);
901
902             t_victf = 0;
903             paso_victf = 0;

```

```

904         animf = 0;
905
906         for(i=0;i<N_ESTRELLAS;i++){
907             star_x[i] = (unsigned char)(4 + (lfsr_siguiente() % 120));
908             if((lfsr_siguiente() & 1) == 0) star_y[i] = (unsigned char)(8 + (
lfsr_siguiente() % 22));
909             else star_y[i] = (unsigned char)(85 + (
lfsr_siguiente() % 35));
910             star_big[i] = (unsigned char)(lfsr_siguiente() & 1);
911             star_on[i] = 0;
912         }
913     }
914
915     t_victf++;
916
917     if(t_victf == 1){
918         if(melodia_victf[paso_victf] == 0) apaga_sonido();
919         else suena_hz(melodia_victf[paso_victf]);
920     }
921
922     if(t_victf >= duracion_victf[paso_victf]){
923         t_victf = 0;
924         paso_victf++;
925         if(paso_victf >= N_VICTF) paso_victf = 0;
926     }
927
928     if((tms % 2) == 0){
929         unsigned char k = (unsigned char)(lfsr_siguiente() % N_ESTRELLAS);
930         if(star_on[k]){
931             dibuja_estrella(star_x[k], star_y[k], star_big[k], GRAPHICS_COLOR_BLACK)
;
932             star_on[k] = 0;
933         }else{
934             star_big[k] = (unsigned char)(star_big[k] ^ 1);
935             dibuja_estrella(star_x[k], star_y[k], star_big[k], GRAPHICS_COLOR_YELLOW
);
936             star_on[k] = 1;
937         }
938         animf++;
939         if(animf >= 4) animf = 0;
940     }
941
942     if(start){
943         start = 0;
944         modo_rapido = 1; // Aumentamos velocidad
945         tiempo = (unsigned int)(tiempo_base/2);
946
947         T = (unsigned int)(tiempo / 25);
948         if(T < 2) T = 2;
949         T4 = (unsigned int)(T / 4);
950         if(T4 < 1) T4 = 1;
951         T2 = (unsigned int)(2 * T);
952
953         semilla((uint16_t)(contador_ticks ^ 0xBEEF));
954         for(i=0;i<32;i++) secuencia[i] = aleatorio_1_4();
955
956         ronda = 1; puntuacion = 0;
957         paso_maquina = 0; paso_jugador = 0;
958         sub_maquina = 0; seleccion = 0;
959
960         estado = MENSAJE_RONDA;
961         tms = 0;
962         apaga_sonido();

```

```

963     }
964     break;
965
966     // Estado de game over
967     case FIN:
968         if(tms == 1) {
969             char cad[24];
970             apaga_sonido();
971
972             if(modos_rapido){
973                 modos_rapido = 0;
974                 tiempo = tiempo_base;
975                 T = (unsigned int)(tiempo / 25); if(T < 2) T = 2;
976                 T4 = (unsigned int)(T / 4); if(T4 < 1) T4 = 1;
977                 T2 = (unsigned int)(2 * T);
978             }
979
980             Graphics_clearDisplay(&g_sContext);
981             Graphics_setForegroundColor(&g_sContext, GRAPHICS_COLOR_WHITE);
982             Graphics_drawStringCentered(&g_sContext, "GAME OVER", -1, 64, 45,
TRANSPARENT_TEXT);
983             sprintf(cad, "PUNTOS: %d", puntuacion);
984             Graphics_drawStringCentered(&g_sContext, (int8_t*)cad, -1, 64, 70,
TRANSPARENT_TEXT);
985             Graphics_drawStringCentered(&g_sContext, "PULSA START", -1, 64, 95,
TRANSPARENT_TEXT);
986
987             suena_hz(150); // Sonido derrota
988         }
989         if(tms >= (600/25)) apaga_sonido();
990
991         if(start) {
992             start = 0;
993             estado = BIENVENIDA;
994             tms = 0;
995         }
996         break;
997     }
998 }
999 }
1000 }

```

Listing 1: Complete code for the Simon Says game (main.c)